Phase 2 Features — Configurable Constraints & Infeasibility Simulation

This document explains how to use the two Phase 2 features. Both are **additive** and safe: the validated solver logic and data-processing stages are untouched, and the simulation tools are strictly read-only.

Feature 1 — Configurable solver constraints

What it does

Lets you tune the solver's **soft** (preference) constraints without touching code. You can enable/disable each constraint and adjust its weight, or apply one of three preset profiles.

Tunable soft constraints:

Key	Meaning	Default weight
`semester_anchor_first`	Pull each group's first session towards the start of its window	100
`semester_anchor_last`	Pull each group's last session towards the end of its window	100
`spacing`	Keep a group's sessions evenly spaced	200
`parity`	Alternate odd/even weeks between parallel groups	50

> The **hard** constraints C1 (no time overlap), C4 (weeks ≥ practice credits) and C5 (no double lab the same day) are always enforced and are **not** configurable. The reserved-slot penalty is also kept fixed (correctness).

Preset profiles

Profile	Intent	first / last / spacing / parity
Strict	Push hard on regularity and anchoring	300 / 300 / 500 / 150
Balanced	Historical defaults (unchanged behaviour)	100 / 100 / 200 / 50
Relaxed	Only anchor loosely; spacing & parity off	30 / 30 / off / off

The configuration file

`config/solver_constraints.yaml`

```yaml
active_profile: Balanced
soft_constraints:
  semester_anchor_first: {enabled: true, weight: 100}
  semester_anchor_last:  {enabled: true, weight: 100}
  spacing:               {enabled: true, weight: 200}
  parity:                {enabled: true, weight: 50}
```

- **Missing or invalid file** → the app falls back to the `Balanced` defaults, so the solver behaves exactly as before Phase 2.
- A weight of `0` or `enabled: false` removes that preference from the objective.
- Valid weight range: `0 .. 100000`. Weights `>= 10000` trigger an "extreme configuration" warning in the UI.

Using the UI

Open the app (`streamlit run app.py`) and select **Configuration Solveur** in the sidebar:
1. Click a profile button (Strict / Equilibre / Detendu) to apply a preset.
2. Fine-tune individual constraints with the toggles and sliders.
3. Review the live preview (effective weights + enabled state).
4. Tick the confirmation box and click **Enregistrer la configuration**.

The new configuration is picked up automatically on the next optimization run. Each run records the applied configuration under the `constraint_config` key of every entry in `reports/solver_stats.json`.

Using it from code

```python
import solver_config as sc

cfg = sc.load_config()                 # validated, with defaults merged in
sc.get_weight(cfg, "spacing")          # -> 200 (0 if disabled)
strict = sc.apply_profile("Strict")    # build a preset config
sc.save_config(strict)                 # validate then write the YAML
```

Feature 2 — Infeasibility simulation (What-If)

What it does

A read-only “what-if” tool to explore how feasibility would change if you **excluded groups** or **added room/time-slot capacity** — without running or altering the real optimization. It reuses the same capacity model as the solver’s infeasibility diagnostic.

Using the UI

Open **Simulateur Infaisabilite** in the sidebar. Three sections:

1. **Exclure des groupes** — select groups to drop, run the simulation, and see the feasibility change, overflow reduction, removed sessions and affected students. A real CP-SAT feasibility dry-run confirms the verdict.
2. **Ajouter des ressources** — enter an extra `(room, day, block, weeks)` slot and test whether it relieves the bottlenecks.
3. **Suggestions automatiques** — analyse detected bottlenecks and propose which groups to exclude or which resources to add, with one-click apply.

All results are clearly labelled as **estimations** and never modify real data.

Using it from code

```
import simulation_engine as se

# sessions: list of dicts with group_id, day_idx, block_id, rooms,
# min_week, max_week, nb_students, session, subject
r1 = se.simulate_without_groups(sessions, ["Fisica|1|S1"])
print(r1["diff"]["became_feasible"], r1["affected_students"])

r2 = se.simulate_with_extra_capacity(
    sessions, [{"resource": "Lab Z", "day_idx": 0, "block_id": "b1", "weeks": 3}])

sug = se.suggest_actions(sessions)     # ranked exclude/add suggestions
dry = se.dry_run_feasibility(sessions) # real CP-SAT feasibility check
```

Data sources (read-only)

- `reports/unplaced_students.json`
- `reports/solver_stats.json`
- `reports/infeasibility_S*.txt`
- `group_composition.csv` (to reconstruct the hypothetical session list)

Tests

- `tests/test_solver_config.py` — configuration loading, validation, presets, save/reload round-trip, extreme detection.
- `tests/test_simulation_engine.py` — bottleneck analysis, exclude/add-capacity simulations, suggestions, CP-SAT dry-run, report parsing.

Run them with:

```
python -m pytest tests/test_solver_config.py tests/test_simulation_engine.py -q
```

Compatibility

- New dependency: `PyYAML` (added to `requirements.txt`).
- Frozen build: `solver_config.py`, `simulation_engine.py` and the `pages/` folder are bundled by `LabScheduling.spec`.
- If `PyYAML` or the config file is unavailable, the solver silently uses the Balanced defaults — no crash, identical behaviour to Phase 1.